TECNICATURA UNIVERSITARIA
EN PROGRAMACIÓN
A DISTANCIA



Metodología de Sistemas II

Trabajo Práctico
Git avanzado

Farias, Gustavo
Matrícula: 101662
Comisión: M2025-13
Repositorio GitHub: Lucenear/Tecnicatura_Programacion_UTN

Parte 1: La "Fisica" de Git (Teoria y Conceptos)

1- El Modelo de Objetos:

Entiendo que los blobs guardan el contenido crudo de mis archivos, sin nombres ni rutas. Los trees funcionan como las carpetas de mi proyecto, guardan los nombres de los archivos, permisos y apuntan a los blobs o a otros trees. Los commits son las instantaneas que tomo, apuntan a un tree principal y guardan metadatos como mi autor, el mensaje y el commit padre. Todos se relacionan creando una estructura de carpetas y archivos en un momento exacto.

2- Direccionamiento por Contenido:

Yo entiendo que Git no guarda mis archivos por su nombre o ruta, sino que calcula un hash de su contenido para generar su identificador unico. Si el contenido cambia, el hash cambia. El hash me garantiza la integridad porque cualquier alteracion minima cambia el identificador, y me ayuda a la deduplicacion porque si dos archivos son iguales, comparten el mismo blob en el disco, ahorrando espacio.

3- El Grafo Dirigido Aciclico (DAG):

Cada commit que hago guarda el hash de su commit padre. Si modifico algo en el pasado, el hash de ese commit cambia, lo que rompe el enlace con su hijo, obligandome a recalcular toda la cadena hacia adelante. Lo veo como una cadena de custodia criptografica donde cada eslabon depende matematicamente del anterior, haciendo imposible alterar el historico sin que se note.

Parte 2: Areas de Trabajo y Flujo Atomico

4- Las Tres Areas:

Cuando hago git add, el contenido del archivo en mi directorio de trabajo se comprime, se le calcula el hash y se guarda un blob en la base de datos. Luego, se actualiza el area de preparacion (staging area) para que apunte a ese nuevo blob. Al hacer git commit, Git toma el estado del staging area, crea un nuevo tree y un nuevo commit que apunta a ese tree, registrandolo oficialmente en mi repositorio.

5- El rol de HEAD:

Para mi, HEAD es un puntero que indica en que rama o commit estoy parado actualmente. Cuando esta "detached" o despegado, HEAD apunta directamente a un commit especifico en lugar de a una rama. Esto me resulta util para explorar el pasado o probar cosas sin mover las ramas principales, manteniendo la resiliencia del proyecto ya que no altero el flujo de trabajo normal.

Parte 3: Gestion y Cirugia del Historico

6- Limpieza de Historico:

Para unirlos yo usaria git rebase -i HEAD~5 y elijo la opcion squash, o un git reset --soft HEAD~5 seguido de un commit nuevo. La diferencia que veo es que --amend solo me sirve para modificar o agregar cambios al ultimo commit realizado, mientras que el rebase interactivo me permite reescribir, unir o reordenar varios commits hacia atras en el historico.

7- Manipulacion Quirurgica:

Yo prefiero cherry-pick cuando solo necesito traer un commit especifico de otra rama sin arrastrar todo el historico de esa rama. Por ejemplo, en un hotfix: si arregle un error critico en la rama de produccion y quiero aplicar solo ese arreglo exacto a la rama de desarrollo sin mezclar otras cosas, uso cherry-pick para copiar ese unico commit.

Parte 4: Casos Practicos de Resiliencia (Resolucion de Problemas)

8- Escenario de Desastre:

No, no esta perdido. Git guarda los objetos en su base de datos local y no los borra de inmediato, solo pierde la referencia visible desde las ramas.
Primero ejecuto git reflog para ver el historico de movimientos de HEAD y encontrar el hash del commit "borrado". Luego, aplico git reset --hard hash o git checkout hash para volver a ese estado y recuperar mi trabajo.

9- Reset vs. Revert:

Para mi, usar reset en codigo ya compartido es mala practica porque reescribe el historico publico. Si mis compañeros ya tienen esos commits, sus repositorios locales chocaran al intentar sincronizar. La alternativa segura que uso es git revert. Este comando no borra nada, sino que crea un commit nuevo que hace exactamente lo contrario al commit erroneo, manteniendo el historico intacto y seguro para el equipo.

Parte 5: Diagnostico y Auditoria

10- Comandos de Bajo Nivel (Plumbing):

Para verificar el tipo de objeto (si es blob, tree o commit) yo usaria git cat-file -t hash. Para visualizar el contenido puro de un tree o un blob en la terminal yo usaria git cat-file -p hash.